

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Комп'ютерна графіка та обробка зображень»

Робота №8

Тема «Точкові та прожекторні світильники»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Туленков А.В.

2025

ЗАВДАННЯ

Завдання на роботу зі сторінки 29:

1. Застосуйте різноманітне освітлення до об'єкта, змініть форму об'єкта.
2. Наведіть алгоритм встановлення освітлення.
3. Наведіть коментарі у коді.

РЕЗУЛЬТАТ

```
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#include <stdio.h>
#include <math.h> // Для sin/cos, хоча glfwGetTime() повертає double

// Функція завантаження текстури за допомогою stb_image
unsigned int loadTexture(const char *path)
{
    unsigned int textureID;
    glGenTextures(1, &textureID);
    int width, height, nrChannels;
    // Завантажуємо дані зображення, 0 означає 'необхідна кількість каналів'
    unsigned char *data = stbi_load(path, &width, &height, &nrChannels, 0);
    if (data)
    {
        GLenum format = GL_RGB;
        // Визначаємо формат на основі кількості каналів
        if (nrChannels == 1)
            format = GL_RED;
        else if (nrChannels == 3)
            format = GL_RGB;
        else if (nrChannels == 4)
            format = GL_RGBA;

        glBindTexture(GL_TEXTURE_2D, textureID);
        // Завантажуємо текстуру
        glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0, format, GL_UNSIGNED_BYTE, data);
        glGenerateMipmap(GL_TEXTURE_2D); // Генеруємо міпмапи
```

```

        // Встановлення параметрів фільтрації
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);

        stbi_image_free(data); // Звільняємо пам'ять
    }
    else
    {
        fprintf(stderr, "Texture failed to load at path: %s\n", path);
        stbi_image_free(data);
    }
    return textureID;
}

int main()
{
    // Ініціалізація GLFW
    if (!glfwInit())
    {
        fprintf(stderr, "GLFW initialization failed\n");
        return -1;
    }

    // Конфігурація GLFW: версія OpenGL 3.3 Core Profile
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE); // Необхідно
для MacOS

    // Створення вікна GLFW
    GLFWwindow *window = glfwCreateWindow(800, 600, "SpotLight
Example", NULL, NULL);
    if (!window)
    {
        fprintf(stderr, "GLFW window creation failed\n");
        glfwTerminate();
        return -1;
    }

    // Налаштування контексту OpenGL
    glfwMakeContextCurrent(window);

    // Ініціалізація GLEW
    if (glewInit() != GLEW_OK)
    {
        fprintf(stderr, "GLEW initialization failed\n");
        glfwDestroyWindow(window);
        glfwTerminate();
        return -1;
    }

    // Включення тестування глибини для коректного рендерингу 3D
    glEnable(GL_DEPTH_TEST);

```

```

glViewport(0, 0, 800, 600);

// Шейдерний код вершинного шейдера
const char *vertexShaderSource = R"(
    #version 330 core
    layout (location = 0) in vec3 aPos;          // Позиція вершини
    layout (location = 1) in vec2 aTexCoord;      // Текстурні координати
    layout (location = 2) in vec3 aNormal;       // Вектор нормалі

    out vec2 TexCoord;
    out vec3 FragPos;    // Позиція фрагмента у світових координатах
    out vec3 Normal;     // Нормаль фрагмента у світових координатах

    uniform mat4 model;    // Модельна матриця
    uniform mat4 view;     // Матриця виду
    uniform mat4 projection; // Матриця проєкції

    void main() {
        // Кінцева позиція вершини
        gl_Position = projection * view * model * vec4(aPos, 1.0);
        TexCoord = aTexCoord;
        // Обчислення позиції фрагмента у світових координатах
        FragPos = vec3(model * vec4(aPos, 1.0));
        // Обчислення нормалі у світових координатах (забезпечує
коректність при масштабуванні)
        Normal = mat3(transpose(inverse(model))) * aNormal;
    }
)";

// Шейдерний код фрагментного шейдера
const char *fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;

    in vec2 TexCoord;
    in vec3 FragPos;
    in vec3 Normal;

    // Структура матеріалу (текстури та блиск)
    struct Material {
        sampler2D diffuse;
        sampler2D specular;
        float shininess;
    };

    // Структура SpotLight (прожектор)
    struct Light {
        vec3 position;
        vec3 direction;
        vec3 ambient;
        vec3 diffuse;
        vec3 specular;
        float constant;
        float linear;
        float quadratic;
        float cutOff;    // Косинус внутрішнього кута
        float outerCutOff; // Косинус зовнішнього кута
    };

```

```

uniform Material material;
uniform Light light;
uniform vec3 viewPos; // Позиція камери

void main() {
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(light.position - FragPos); //
Вектор світла до фрагмента
    vec3 viewDir = normalize(viewPos - FragPos); //
Вектор камери до фрагмента

    // 1. Ambient (Фонове) Освітлення
    vec3 ambient = light.ambient * texture(material.diffuse,
TexCoord).rgb;

    // 2. Diffuse (Розсіяне) Освітлення
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = light.diffuse * diff *
texture(material.diffuse, TexCoord).rgb;

    // 3. Specular (Дзеркальне) Освітлення
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0),
material.shininess);
    vec3 specular = light.specular * spec *
texture(material.specular, TexCoord).rgb;

    // 4. Attenuation (Згасання з відстанню)
    float distance = length(light.position - FragPos);
    float attenuation = 1.0 / (light.constant + light.linear *
distance + light.quadratic * (distance * distance));

    // 5. SpotLight (Обмеження конусом)
    vec3 spotDir = normalize(light.direction);
    // theta - косинус кута між вектором світла та напрямком
прожектора.
    // -spotDir використовується, тому що lightDir спрямований
від світла до фрагмента, а не навпаки.
    float theta = dot(lightDir, -spotDir);

    float intensity = 0.0;
    // Перевіряємо, чи знаходиться фрагмент всередині
зовнішнього конуса
    if (theta > light.outerCutoff)
    {
        // Якщо всередині внутрішнього конуса, освітлення повне
(1.0)
        if (theta > light.cutoff)
            intensity = 1.0;
        else // Інакше - плавний перехід
        {
            float epsilon = light.cutoff - light.outerCutoff;
            // Плавний перехід від 0.0 до 1.0
            intensity = clamp((theta - light.outerCutoff) /
epsilon, 0.0, 1.0);
        }
    }
}

```

```

        // 6. Final Color (Остаточний колір)
        // Застосовуємо інтенсивність SpotLight та згасання до
diffuse/specular
        vec3 lighting = ambient + intensity * (diffuse + specular);
        vec3 result = lighting * attenuation;

        FragColor = vec4(result, 1.0);
    }
}";

// Компіляція і лінкування шейдерів
GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
glCompileShader(vertexShader);

GLuint fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);

GLuint shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
glUseProgram(shaderProgram);

glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

// Вершини, текстурні координати та нормалі ПЛОЩИНИ (для
// демонстрації SpotLight)
// Розмір: 2x2. Нормаль: (0, 1, 0)
GLfloat vertices[] = {
    // Позиції           // Текстурні координати // Нормалі
    -1.0f, 0.0f, -1.0f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f, // 1
    1.0f, 0.0f, -1.0f, 1.0f, 1.0f, 0.0f, 1.0f, 0.0f, // 2
    1.0f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f, // 3

    1.0f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f, // 3
    -1.0f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, // 4
    -1.0f, 0.0f, -1.0f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f // 1
};

// Створення буферів та об'єктів
GLuint VAO, VBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);

// Зв'язка об'єктів
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

// Вершинні атрибути (3+2+3 = 8 елементів на вершину)
// 0: Позиція (3 float)
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 *
sizeof(GLfloat), (GLvoid *)0);

```

```

    glEnableVertexAttribArray(0);
    // 1: Текстурні координати (2 float)
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 8 *
sizeof(GLfloat), (GLvoid *) (3 * sizeof(GLfloat)));
    glEnableVertexAttribArray(1);
    // 2: Нормалі (3 float)
    glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 8 *
sizeof(GLfloat), (GLvoid *) (5 * sizeof(GLfloat)));
    glEnableVertexAttribArray(2);

    // Зняття зв'язку
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);

    // Завантаження текстур (використовуйте шляхи до ваших файлів
texture3.jpg та texture4.jpg)
    unsigned int diffuseMap = loadTexture("texture3.jpg");
    unsigned int specularMap = loadTexture("texture4.jpg");

    // Встановлення юніформів текстур (одноразово)
    glUseProgram(shaderProgram);
    glUniform1i(glGetUniformLocation(shaderProgram,
"material.diffuse"), 0);
    glUniform1i(glGetUniformLocation(shaderProgram,
"material.specular"), 1);

    // Параметри світла (SpotLight)
    glm::vec3 lightPos(0.0f, 1.5f, 0.0f);          // Початкова позиція
світла
    glm::vec3 lightDir(0.0f, -1.0f, 0.0f);        // Світло спрямоване
строго вниз
    glm::vec3 lightAmbient(0.1f, 0.1f, 0.1f);    // Слабо фонове світло
    glm::vec3 lightDiffuse(0.8f, 0.8f, 0.8f);
    glm::vec3 lightSpecular(1.0f, 1.0f, 1.0f);
    float lightConstant = 1.0f;
    float lightLinear = 0.07f;
    float lightQuadratic = 0.017f;
    // Кути конуса (перетворюємо градуси в косинус)
    float lightCutOff = glm::cos(glm::radians(10.0f));
    float lightOuterCutOff = glm::cos(glm::radians(15.0f));

    // Параметри матеріалу
    float materialShininess = 32.0f;

    // Позиція камери
    glm::vec3 cameraPos(0.0f, 3.0f, 5.0f);

    // Основний цикл рендерингу
    while (!glfwWindowShouldClose(window))
    {
        // Отримання часу для динамічного руху світла
        float time = (float)glfwGetTime();

        // Динамічний рух SpotLight по X і Z
        lightPos.x = sin(time * 0.5f) * 3.0f;
        lightPos.z = cos(time * 0.5f) * 3.0f;

```

```

        glClearColor(0.1f, 0.1f, 0.1f, 1.0f); // Темний фон, щоб
        підкреслити SpotLight
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

        glUseProgram(shaderProgram);

        // Встановлення матриць трансформації
        glm::mat4 model = glm::mat4(1.0f);
        model = glm::scale(model, glm::vec3(5.0f)); // Збільшення
        площини в 5 разів

        glm::mat4 view = glm::lookAt(cameraPos, glm::vec3(0.0f, 0.0f,
        0.0f), glm::vec3(0.0f, 1.0f, 0.0f));
        glm::mat4 projection = glm::perspective(glm::radians(45.0f),
        800.0f / 600.0f, 0.1f, 100.0f);

        // Отримання локацій uniform-змінних
        GLuint modelLoc = glGetUniformLocation(shaderProgram, "model");
        GLuint viewLoc = glGetUniformLocation(shaderProgram, "view");
        GLuint projectionLoc = glGetUniformLocation(shaderProgram,
        "projection");
        GLuint viewPosLoc = glGetUniformLocation(shaderProgram,
        "viewPos");
        GLuint lightPosLoc = glGetUniformLocation(shaderProgram,
        "light.position");
        GLuint lightDirLoc = glGetUniformLocation(shaderProgram,
        "light.direction");
        GLuint lightAmbientLoc = glGetUniformLocation(shaderProgram,
        "light.ambient");
        GLuint lightDiffuseLoc = glGetUniformLocation(shaderProgram,
        "light.diffuse");
        GLuint lightSpecularLoc = glGetUniformLocation(shaderProgram,
        "light.specular");
        GLuint lightConstantLoc = glGetUniformLocation(shaderProgram,
        "light.constant");
        GLuint lightLinearLoc = glGetUniformLocation(shaderProgram,
        "light.linear");
        GLuint lightQuadraticLoc = glGetUniformLocation(shaderProgram,
        "light.quadratic");
        GLuint lightCutoffLoc = glGetUniformLocation(shaderProgram,
        "light.cutoff");
        GLuint lightOuterCutoffLoc =
        glGetUniformLocation(shaderProgram, "light.outerCutoff");
        GLuint materialShininessLoc =
        glGetUniformLocation(shaderProgram, "material.shininess");

        // Передача uniform-змінних
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
        glm::value_ptr(model));
        glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
        glUniformMatrix4fv(projectionLoc, 1, GL_FALSE,
        glm::value_ptr(projection));
        glUniform3fv(viewPosLoc, 1, glm::value_ptr(cameraPos));

        // Передача динамічних параметрів світла
        glUniform3fv(lightPosLoc, 1, glm::value_ptr(lightPos));
        glUniform3fv(lightDirLoc, 1, glm::value_ptr(lightDir));
        glUniform3fv(lightAmbientLoc, 1, glm::value_ptr(lightAmbient));

```



```

        glUniform3fv(lightDiffuseLoc, 1, glm::value_ptr(lightDiffuse));
        glUniform3fv(lightSpecularLoc, 1,
glm::value_ptr(lightSpecular));
        glUniform1f(lightConstantLoc, lightConstant);
        glUniform1f(lightLinearLoc, lightLinear);
        glUniform1f(lightQuadraticLoc, lightQuadratic);
        glUniform1f(lightCutOffLoc, lightCutOff);
        glUniform1f(lightOuterCutOffLoc, lightOuterCutOff);
        glUniform1f(materialShininessLoc, materialShininess);

        // Активація текстур
        glActiveTexture(GL_TEXTURE0);
        glBindTexture(GL_TEXTURE_2D, diffuseMap);
        glActiveTexture(GL_TEXTURE1);
        glBindTexture(GL_TEXTURE_2D, specularMap);

        // Малювання площини (6 вершин)
        glBindVertexArray(VAO);
        glDrawArrays(GL_TRIANGLES, 0, 6);
        glBindVertexArray(0);

        // Обмін буферів та обробка подій
        glfwSwapBuffers(window);
        glfwPollEvents();
    }

    // Вивільнення ресурсів
    glDeleteVertexArrays(1, &VAO);
    glDeleteBuffers(1, &VBO);
    glDeleteTextures(1, &diffuseMap);
    glDeleteTextures(1, &specularMap);

    // Завершення роботи GLFW
    glfwDestroyWindow(window);
    glfwTerminate();
    return 0;
}

```